

1. Sistema de Reservas de Hotel

Objetivo: Gerenciar o fluxo de ocupação e cadastro de quartos de um hotel.

- **Estrutura (struct):** Quarto contendo numero (int), tipo (string, ex: "Solteiro" ou "Casal"), preco_diaria (float) e ocupado (int - 1 para sim, 0 para não).
- **Alocação Dinâmica:** O sistema inicia alocando um número padrão de quartos e permite adicionar novos quartos à estrutura do hotel conforme a demanda usando realloc.
- **Operações:**
 1. **Inserção:** Cadastrar novos quartos no sistema.
 2. **Consulta:** Buscar quartos por número ou filtrar por disponibilidade.
 3. **Alteração:** Realizar check-in/check-out (alterar o status do quarto).
 4. **Relatórios:** Listar taxa de ocupação do hotel (%) e faturamento potencial com os quartos ocupados.

2. Gerenciador de Eventos e Ingressos

Objetivo: Controlar a venda e validação de ingressos para shows e conferências.

- **Estrutura (struct):** Ingresso contendo id (int), nome_comprador (string), categoria (string, ex: "Pista", "VIP") e validado (int - 1 para sim, 0 para não).
- **Alocação Dinâmica:** Alocar dinamicamente a lista de ingressos vendidos conforme as compras acontecem.
- **Operações:**
 1. **Inserção:** Realizar a venda de um novo ingresso realocando a memória.
 2. **Consulta:** Consultar um ingresso pelo ID para verificar a quem pertence.
 3. **Alteração:** Validar a entrada do participante no evento (mudar o status para "utilizado").
 4. **Relatórios:** Listar quantidade total de ingressos vendidos por categoria e quantos já entraram no evento.

3. Controle de Manutenção de Equipamentos

Objetivo: Registrar chamados de assistência técnica para equipamentos de TI em uma empresa.

- **Estrutura (struct):** Equipamento contendo tombamento (int - código de patrimônio), descricao (string), setor (string) e status (int - 0 para Operacional, 1 para Em Manutenção).
- **Alocação Dinâmica:** Criar o vetor dinâmico de equipamentos através de ponteiros e gerenciar o redimensionamento dinâmico.
- **Operações:**

1. **Inserção:** Cadastrar um novo equipamento no parque de TI.
2. **Consulta:** Buscar equipamento pelo número de tombamento ou setor.
3. **Alteração:** Alterar o status do equipamento (enviar para manutenção ou liberar para uso).
4. **Relatórios:** Relatório de equipamentos que estão atualmente em manutenção e distribuição por setor.

4. Plataforma de Streaming de Músicas

Objetivo: Simular um catálogo de músicas de uma playlist ou serviço de áudio.

- **Estrutura (struct):** Musica contendo id (int), titulo (string), artista (string), duracao_segundos (int) e visualizacoes (int).
- **Alocação Dinâmica:** O vetor dinâmico é gerenciado via funções com ponteiro para ponteiro (Musica **catalogo) para inserção e remoção de registros.
- **Operações:**
 1. **Inserção:** Adicionar uma nova música ao catálogo da plataforma.
 2. **Consulta:** Buscar músicas por título ou por nome do artista.
 3. **Alteração:** Incrementar o número de visualizações/reproduções de uma música específica.
 4. **Relatórios:** Listar o "Top Músicas" mais ouvidas e a duração total acumulada do catálogo em minutos.

5. Sistema de Gestão de Estoque de Alimentos

Objetivo: Criar um controle para uma pequena mercearia.

- **Estrutura (struct):** Produto contendo codigo (int), nome (string), quantidade (int) e preco (float).
- **Alocação Dinâmica:** O sistema deve começar alocando memória para um número inicial de produtos (ex: 2) e utilizar realloc para expandir a memória dinamicamente a cada novo cadastro.
- **Operações:**
 1. **Inserção:** Adicionar um novo produto realocando a memória do vetor de structs.
 2. **Consulta:** Buscar um produto pelo código ou por nome e exibir seus dados.
 3. **Alteração:** Atualizar a quantidade em estoque ou o preço de um produto cadastrado.
 4. **Relatórios:** Listar todos os produtos e gerar um relatório mostrando produtos com estoque abaixo do limite mínimo definido.

6. Biblioteca Digital de Livros

Objetivo: Administrar o acervo de livros de um clube de leitura.

- **Estrutura (struct):** Livro contendo id (int), titulo (string), autor (string) e disponivel (int - 1 para sim, 0 para não).
- **Alocação Dinâmica:** Gerenciar um ponteiro para ponteiro ou um vetor dinâmico que redimensiona dinamicamente conforme os livros são inseridos e removidos.
- **Operações:**
 1. **Inserção:** Cadastrar um novo livro no acervo com status inicial "Disponível".
 2. **Consulta:** Buscar livros pelo título ou por nome do autor.
 3. **Alteração:** Alterar o status do livro (emprestado/devolvido).
 4. **Relatórios:** Mostrar a listagem de todos os livros emprestados e o total acumulado do acervo.

7. Sistema de Adoção de Animais

Objetivo: Gerenciar os animais abrigados em uma ONG até serem adotados.

- **Estrutura (struct):** Pet contendo id (int), nome (string), especie (string, ex: "Cão", "Gato"), idade (int) e adotado (int - 1 para sim, 0 para não).
- **Alocação Dinâmica:** Inicia com alocação para um número fixo e usa realloc sempre que um novo animal der entrada no abrigo.
- **Operações:**
 1. **Inserção:** Cadastrar um novo animal resgatado.
 2. **Consulta:** Buscar por ID ou filtrar animais por espécie.
 3. **Alteração:** Atualizar o status do animal para "Adotado" ou atualizar a idade.
 4. **Relatórios:** Listar apenas os animais ainda disponíveis para adoção e exibir o total de adoções realizadas.

8. Controle de Manutenção de Bicicletas (Oficina)

Objetivo: Gerenciar a ordem de serviço (O.S.) em uma oficina de reparos.

- **Estrutura (struct):** OrdemServico contendo numero_os (int), cliente (string), descricao_defeito (string) e valor_orcamento (float).
- **Alocação Dinâmica:** Gerenciamento do vetor dinâmico de Ordens de Serviço, garantindo também a liberação de memória (free) ao fechar o sistema.
- **Operações:**
 1. **Inserção:** Abrir uma nova Ordem de Serviço.
 2. **Consulta:** Pesquisar O.S. pelo número ou pelo nome do cliente.

3. **Alteração:** Atualizar o valor do orçamento ou adicionar observações ao conserto.
4. **Relatórios:** Listar ordens de serviço acima de determinado valor e calcular o valor médio gasto por reparo na oficina.

9. Monitor de Descarte e Reciclagem Inteligente

Objetivo: Controlar pontos de coleta seletiva e o volume de resíduos acumulados.

- **Estrutura (struct):** PontoColeta contendo id (int), localizacao (string), tipo_residuo (string, ex: "Plástico", "Eletrônicos") e capacidade_ocupada (float, em %).
- **Alocação Dinâmica:** Aloca e redimensiona a lista de pontos de coleta na cidade à medida que novas lixeiras inteligentes são instaladas.
- **Operações:**
 1. **Inserção:** Cadastrar um novo ponto de coleta no sistema.
 2. **Consulta:** Buscar por ID ou filtrar por tipo de resíduo aceito.
 3. **Alteração:** Atualizar a porcentagem de capacidade ocupada enviada pelos sensores.
 4. **Relatórios:** Listar pontos de coleta críticos (capacidade acima de 80%) para orientar as rotas de recolhimento.

10. Sistema de Bilhetagem para Carregadores de Veículos Elétricos

Objetivo: Controlar pontos de recarga de carros elétricos em um eletroposto.

- **Estrutura (struct):** PontoRecarga contendo id_ponto (int), potencia_kw (float), kwh_consumidos (float) e ocupado (int - 1 para sim, 0 para não).
- **Alocação Dinâmica:** Definir a quantidade inicial de conectores via malloc e permitir a expansão do eletroposto.
- **Operações:**
 1. **Inserção:** Adicionar novos pontos/conectores de recarga ao sistema.
 2. **Consulta:** Consultar a disponibilidade de um ponto específico pelo ID.
 3. **Alteração:** Registrar o início/fim de uma sessão de recarga e atualizar a quantidade de kWh transferidos.
 4. **Relatórios:** Exibir o total de energia fornecida no dia e o faturamento estimado do eletroposto.

11. Gerenciador de Armários Inteligentes (Smart Lockers)

Objetivo: Controlar a entrega e retirada de encomendas em armários compartilhados de condomínios.

- **Estrutura (struct):** Armario contendo numero_gaveta (int), tamanho (string, ex: "P", "M", "G"), codigo_retirada (int) e ocupado (int - 1 para sim, 0 para não).

- **Alocação Dinâmica:** Utilização de vetores dinâmicos onde os dados de cada gaveta são inicializados e gerenciados via ponteiros em funções dedicadas.
- **Operações:**
 1. **Inserção:** Cadastrar novos módulos de armários no prédio.
 2. **Consulta:** Buscar gavetas livres filtradas pelo tamanho necessário para o pacote.
 3. **Alteração:** Depositar uma encomenda (gerando o código de retirada) ou liberar o armário após o resgate.
 4. **Relatórios:** Mostrar a taxa de ocupação dos armários e o tempo médio em que as encomendas ficam aguardando retirada.

12. Plataforma de Troca de Créditos de Logística Reversa

Objetivo: Registrar a devolução de embalagens reutilizáveis por clientes em supermercados e atribuir pontos de benefício.

- **Estrutura (struct):** ClientePontos contendo cpf (string), nome (string), embalagens_devolvidas (int) e saldo_pontos (int).
- **Alocação Dinâmica:** Inicialização com malloc e redimensionamento progressivo conforme novos clientes realizam sua primeira troca de embalagem.
- **Operações:**
 1. **Inserção:** Cadastrar um novo cliente no programa de logística reversa.
 2. **Consulta:** Buscar dados do participante pelo CPF.
 3. **Alteração:** Registrar a devolução de embalagens e somar os pontos acumulados (\$pontos = embalagens \times multiplicador\$).
 4. **Relatórios:** Mostrar o ranking dos clientes com mais devoluções e o total acumulado de embalagens recicladas/reutilizadas.

13. Monitor de Qualidade do Ar Urbano (Sensores IoT)

Objetivo: Gerenciar dados transmitidos por estações de monitoramento de poluição do ar nas cidades.

- **Estrutura (struct):** EstacaoAr contendo id (int), bairro (string), indice_aqi (int - Índice de Qualidade do Ar) e nivel_co2_ppm (float).
- **Alocação Dinâmica:** O vetor dinâmico expande via realloc à medida que novas estações de monitoramento são instaladas na cidade.
- **Operações:**
 1. **Inserção:** Cadastrar uma nova estação de medição de qualidade do ar.
 2. **Consulta:** Buscar dados de poluição por ID da estação ou pelo nome do bairro.
 3. **Alteração:** Atualizar as leituras do índice AQI e nível de CO₂ enviadas pelos sensores.

4. **Relatórios:** Listar bairros com qualidade do ar em nível crítico ($AQI > 100$) e calcular a média de CO_2 da cidade.